UNIVERSIDAD NACIONAL DE COLOMBIA

MSC THESIS · STUDY 1 (V1.2E) RESULTS AND THE V1.3 VALIDITY CYCLE

Governing AI-Agentic Development: A Spec-Driven Neuro-Symbolic Architectural Firewall

Study 1 (v1.2E) results: what holds, what doesn't, and the v1.3 cycle that makes the tool provably valid before Study 2.

Cristian Córdoba
Author

Director: [name]
Thesis director

MSc in Systems and Computing Engineering
Universidad Nacional de Colombia
October 2026

Outline

I **Context and method**
The tool, the specific objectives SO1–SO5, pre-registration and how to read the numbers

II **Study 1 results**
SO2 extraction, SO4 seeded detection, real-code precision, tool comparison, SO3 judge, SO5 E1, plain LLM-review baseline

III **Discussion**
Status of the objectives, what Study 1 supports, threats to validity, contributions

IV **v1.3 validity cycle**
The four-link measurement chain and the first evidence

V **Decisions**
What I need from the director, then references and appendix (detailed SO4 table, pre-registration bumps, rule sources)

I

Context and method

The tool · Specific objectives SO1–SO5 · Pre-registration and reproduction

DaedalusArch turns an architecture spec into checks that run on every change



Symbolic core

Deterministic, auditable and fast. The same code always gets the same findings.

Neural layer

Explains and judges what rules can't express. It's frozen, pinned and recorded.

Use case

A CI gate, or an agent's tool through MCP, that stops architectural erosion as it happens.

Fitness functions in the sense of Ford, Parsons & Kua (2017). Judge: `claude-opus-5-5`, frozen rubric. [Docs/reports/director-report-2026-10.md §1, §3.2, §3.6; Docs/judge-preregistration.md]

Specific objectives SO1–SO5, as promised in the proposal

Table 1. Specific objectives and the experiments of Study 1 that evaluate them.

SO	What it promises	Evaluated in Study 1 by
SO1	Spec language and style libraries (Clean, Hexagonal, Layered) that compile to executable checks	Template coverage per library; layered acceptance run
SO2	APG extraction: coverage, latency, value of the graph over a plain AST	Parse coverage, latency gate, full APG vs AST-only ablation
SO3	Symbolic rules plus a neural judge producing an Architectural Health Score (AHS) and verdicts	E7 (9 projects), judge-vs-panel P4, judge model robustness, judge-catch test B
SO4	P/R/F1 on a seeded golden dataset (80 to 120 instances)	Held-out seeded set on 7 bases, symbolic-only
SO5	LLM × spec-quality study (E1)	E1 grid: generators × spec level × task

Two further comparisons frame the results: dependency-cruiser on the same seeds, and a plain LLM review given the same spec as prose.

[Docs/reports/director-report-2026-10.md §2; Docs/analysis-plan.md §2, §6]

Every result is pre-registered or declared post-hoc, and all of it reproduces

v25

Pre-registration version, with 85 hashed artefacts (plans, rules, rubric, corpus)

Gate

`run-experiment` refuses any plan whose registered artefacts changed, and CI checks it on every PR

ADRs

Every change made after seeing data is an ADR, reported beside the registered reading, never instead of it

310

Result tables reproduced byte for byte by an independent audit, with 0 differences

> **How to read the numbers that follow.** pre-registered is the primary figure; post-hoc is a declared correction, shown next to it.

Each bump of the registration has a dated reason; the full list of bumps is in Appendix A2.

[corpus/prereg.json; Docs/threats-to-validity.md, TV-01, TV-91; Docs/prereg-reasons.md; Docs/DiagnosticRuns/reproduction-audit-2026-10-10.md]

II

Study 1 results

SO2 extraction · SO4 seeded detection · Real-code precision · Comparison with dependency-cruiser · SO3 the LLM judge · SO5 the E1 grid · Plain LLM-review baseline

Extraction is complete and fast enough for CI; data-flow edges are a known gap

100%

Parse coverage on every base (4 real core bases, up to 241 files)

5.3 s

Full evaluation of ghostfolio (241 files). The slowest cycle query took 1.35 s against a 30 s budget. pre-registered

0

FLOWS_TO (data-flow) edges on real NestJS code, because collaborators arrive by injection. Declared limitation (ADR-023).

Table 2. Full APG vs AST-only ablation on the four core bases (instrument v1, pre-registered, descriptive).

Base	Files	Parse cov.	Functions detecting (full → AST-only)	Functions lost
realworld-test	35	1.000	6 → 6	none
ghostfolio-test	241	1.000	13 → 12	FF-P02; FF-SO01 fewer (22 → 16)
truthy-demo	131	1.000	12 → 10	FF-P02, FF-P04
dry-run-test	155	1.000	11 → 9	FF-P02, FF-P04

> The ablation shows why the graph matters. With only AST import edges, the dependency-inversion (FF-P02) and use-case-isolation (FF-P04) rules go blind on every real base where they fired.

[results/apg-ablation/so2/graph_coverage.csv; results/latency-gate/so2/latency.csv; results/latency-gate/so2/gate.json; results/apg-ablation/so2/apg_ablation_summary.csv; results/apg-ablation/so2/flows_to_stores.csv; ADR-023]

On held-out code it finds the violations it was built to find

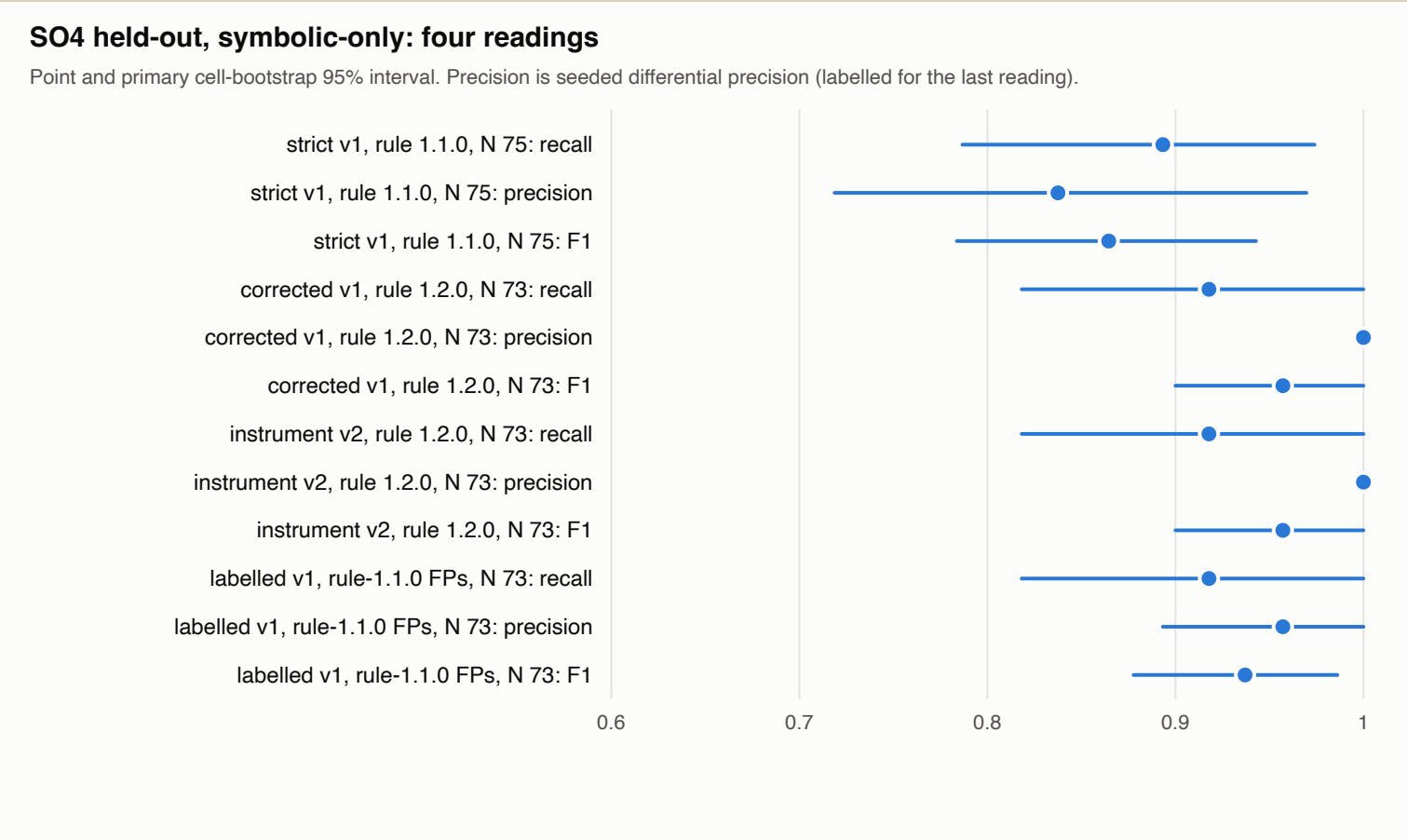


Figure 1. SO4 held-out, symbolic-only: recall, precision and F1 per reading, with cell-bootstrap 95% intervals.

[results/so4-heldout/agg-strict/prf_overall.csv; results/so4-heldout/agg-corrected/prf_overall.csv; results/so4-heldout/agg/prf_overall.csv; results/so4-heldout/v2/agg/prf_overall.csv; built by make_figures.py]

The registered reading comes first; the corrections sit beside it

Table 3. SO4 readings (precision is seeded differential precision).

Reading	Recall	Prec.	F1
Strict, N 75 reg.	0.893	0.838	0.865
Corrected, N 73 post-hoc	0.918	1.000	0.957
Labelled, N 73 post-hoc	0.918	0.957	0.937

Intervals, TP/FP/FN, twin specificity and the instrument-v2 reading: Appendix A1.

- All 6 misses are dynamic `import()`, which is outside extractor coverage. Now fixed in v1.3.
- N = 75 is below the floor of 80, because 10 seeds failed type-check. This is reported as a deviation.
- Recall measures **construct sensitivity**, not detection of naturally occurring violations.

[results/so4-heldout/agg-strict/prf_overall.csv; results/so4-heldout/agg-corrected/prf_overall.csv; results/so4-heldout/score.json; results/so4-heldout/agg/prf_overall.csv; results/labels/fn-causes.json; results/so4-heldout/agg-strict/golden_instances.csv; ADR-024, ADR-025, ADR-033]

On real projects, precision is low and poorly measured; that is the weak point

0.33

Registered baseline precision (instrument v2), Kish-Wilson 95% CI 0.13–0.61 pre-registered

0.33–0.79

The range depends on how the 8 "uncertain" labels are counted, out of 24 items post-hoc

n_{eff} 11.5

Only 24 labelled items: a pilot, not a precision estimate

Why so noisy

Core projects score 0.59 and E7 projects 0.21. The E7 specs were generated **mechanically from folder names**. False alarms come from metric and convention proxies, type-only imports and layer mapping, not from the structural import rules.

What it means

A finding can be wrong at four points: graph, spec, rule or meaning. Study 1 measured only the end-to-end result. **v1.3 measures each point separately**, on greenfield code where the spec is known.

[results/so4-heldout/agg/precision_baseline.csv; results/so4-heldout/agg/fp_fn_taxonomy.csv; Docs/e7-spec-rule.md; ADR-033]

It matches a mature import checker on the rules both tools can express

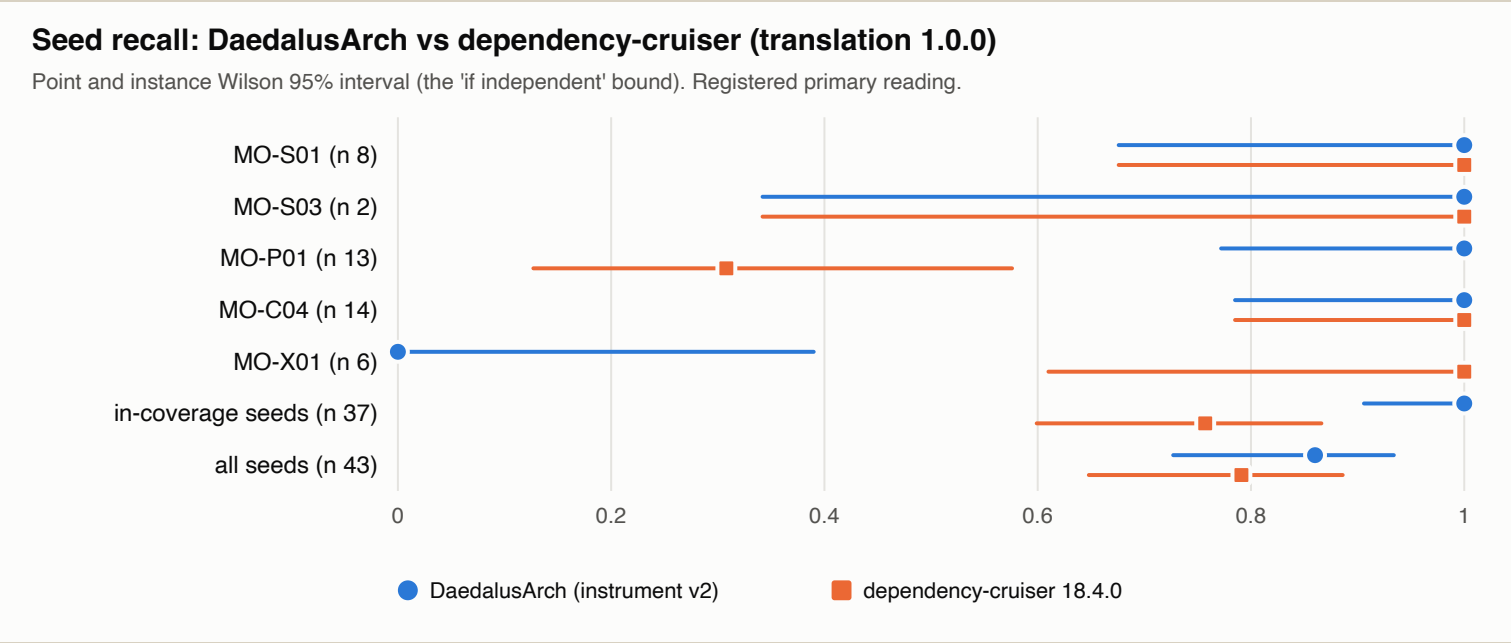


Figure 2. Same seeds, comparable rules: DaedalusArch vs dependency-cruiser 18.4.0.

- **Shared rules:** they agree, apart from two explained points.
- **dependency-cruiser** catches the 6 dynamic-import seeds that we missed.
- Our apparent edge on domain purity came from a translation defect that favoured us. It was declared post-hoc.
- **Real fix commits:** both tools flag the same 2 of 5.
- **Our advantage is breadth:** pattern, SOLID-proxy, metric and judge dimensions that have no external comparator yet.

[results/tool-comparison/recall.csv; results/tool-comparison/recall-dts.csv; results/real-pairs/summary.json; Docs/DiagnosticRuns/tool-comparison-real-pairs-2026-10-10.md; ADR-030]

The judge is consistent and stable, but its effect and its catch rate are unproven

73/331

Judge units failed across the 9 real E7 projects

0

Verdicts changed at the registered 8% weight (an algebraic bound). Variants move 0 to 8 of 9.

0.982

Agreement and AC1 (Gwet, 2008) with a Gemini panel on E1 (n 36). κ is 0 only because almost every item is "pass".

κ 0.766

Sonnet vs Opus as the judge: 94% agreement, 0 verdict changes

Open gap. The judge passed 36 of 36 E1 items, and the planned catch test (B) placed 0 seeds. So we know it doesn't wrongly fail good code, but not whether it catches bad code. v1.3 adds realistic anti-pattern seeds.

[results/e7-corpus/judge-diagnostics/judge_summary.csv; results/e7-corpus/agg-proportional/ahs_by_aggregation.csv; results/e1-grid/label-plan-p4-api/labelling.json, row judge-vs-panel; results/judge-robustness/agreement.csv; Docs/DiagnosticRuns/judge-probe-e1-site-count.md; ADR-034, ADR-035, ADR-037, ADR-038]

Specs change what agents build; a precise spec beats vague prose

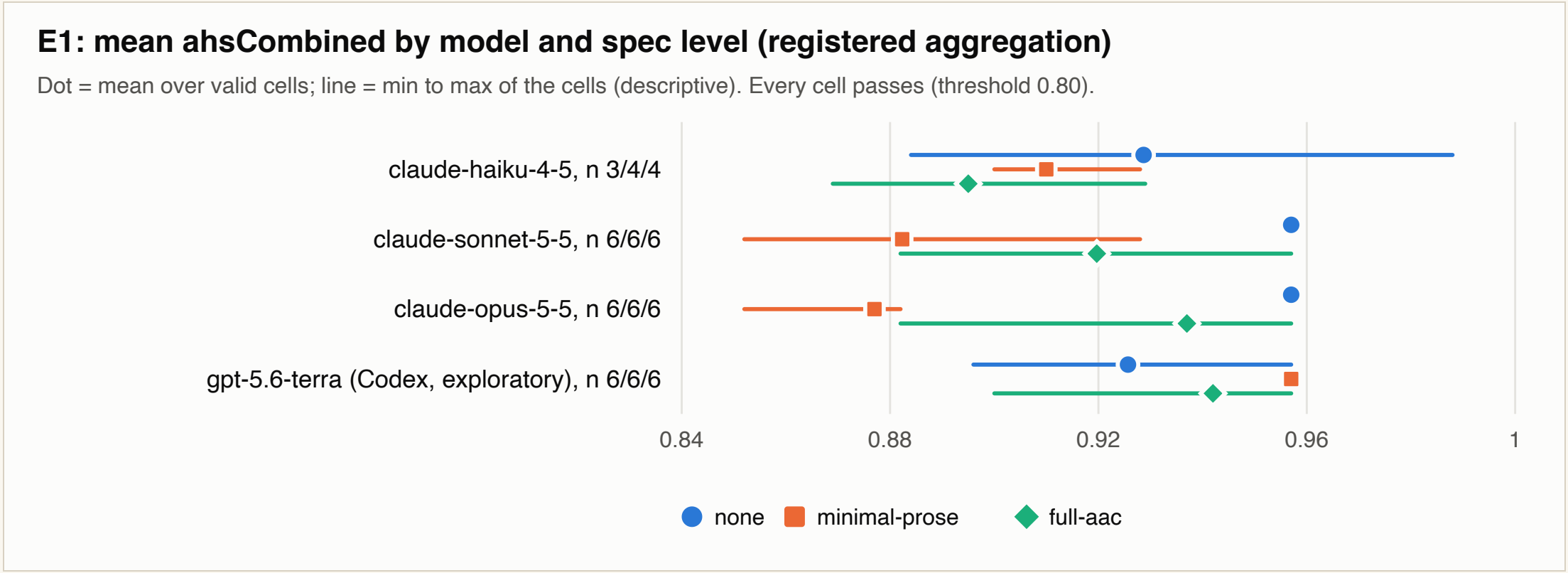


Figure 3. AHS by generator and spec level. 72 cells (3 Claude models + Codex), 65 judged.

[results/e1-grid/agg/so5_grid.csv; results/e1-grid/agg/so5_tests.csv; built by make_figures.py]

The reversed spec effect runs through project size

- Spec level has an effect (Holm p 0.0003), but the direction is **reversed**: no spec scores highest. reg.
- **Why:** without a spec, models build small flat projects (about 9 files) where few rules can fire. Findings per file are equal across levels.
- **At equal size**, a full AaC spec beats minimal prose: Cliff's δ 0.52 (0.47 after LOC adjustment). post-hoc
- No model effect. One rule (test-file pairing) produced 83% of findings.
- Self-preference p 0.014, driven by the Haiku cells.

Table 4. Claude arm (47 valid cells), by spec level.

Spec level	Mean `ahsCombined`	Mean files	Findings per file
`none`	0.951	9.1	1.14
`minimal-prose`	0.887	27.4	1.15
`full-aac`	0.920	29.6	1.15

The size analysis is post-hoc. Every judged cell passes (minimum AHS 0.852), so E1 studies differences inside the pass band.

[results/e1-grid/agg/so5_tests.csv; results/e1-grid/agg/so5_grid.csv; results/e1-grid/agg-size/size_levels.csv; results/e1-grid/agg-size/size_tests.csv; results/e1-grid/agg-size/size_functions.csv; Docs/DiagnosticRuns/e1-size-analysis-2026-10-10.md]

A plain LLM reviewer finds planted bugs, but is not repeatable or exhaustive

9/10

Planted seeds found by Codex GPT-5.6 given the same spec as prose (the firewall found 5/5)

0.727

Run-to-run overlap (Jaccard): the same tree reviewed twice gives different findings

26.5%

Share of the firewall's findings that the reviewer listed

~3 min

and about 6 credits per review, against seconds for the firewall

> The firewall's case is determinism, exhaustiveness, speed and auditability, not better seed detection. (Exploratory pilot: it stopped at its registered budget after 25 of 42 reviews.)

[results/llm-review/recall.csv; results/llm-review/consistency.csv; results/llm-review/overlap.csv; results/llm-review/cost.csv; Docs/DiagnosticRuns/llm-review-pilot-2026-10-11.md; ADR-036]

III

Discussion

Status of SO1–SO5 · What Study 1 supports, and what it does not yet · Threats to validity · Contributions

Status of SO1–SO5 on the frozen Study 1 branch

Table 5. Status of each specific objective on branch `v1.2e`.

SO	Promise	Status	Evidence in one line
SO1	Spec language and style libraries	Partly	Clean, Layered and NestJS: 27/27 templates each. Hexagonal was missing in Study 1 and is now built in v1.3.
SO2	Graph extraction, coverage and latency	Done	100% of files parse; 5.3 s on the largest base against a 30 s budget; FLOWS_TO limitation declared.
SO3	Rules plus judge give AHS and verdicts	Partly	Judge agrees with a Gemini panel (AC1 0.982) and is model-stable (κ 0.766), but it changes no verdict and its catch rate is unmeasured.
SO4	P/R/F1 on a seeded golden set (80–120)	Shortfall	Recall 0.893 registered, 0.918 corrected. N = 75 against the floor of 80. Real-code precision is uncertain.
SO5	LLM × spec-quality study (E1)	Run	72 cells, 65 judged. Spec level matters, but through project size; full spec beats vague prose.

[Docs/reports/director-report-2026-10.md §2; results/pre-tag/so1-metrics-53cfaed4a313.json; results/latency-gate/so2/gate.json; results/e1-grid/label-plan-p4-api/labelling.json; results/judge-robustness/agreement.csv; results/so4-heldout/; results/e1-grid/agg/]

What Study 1 supports, and what it does not yet

Supports

- It finds the canonical violation constructs reproducibly, on held-out code, at CI speed.
- It matches dependency-cruiser on the rules both can express.
- Specs change the structure agents build, the AHS registers it, and vague prose is worst.
- The judge isn't failing sound code, and it's stable across judge models.
- The method holds: pre-registration, declared post-hoc changes, and byte-identical reproduction.

Does not support yet

- That findings on real code are mostly correct (0.33 registered, wide CI).
- That the judge improves verdicts or catches unsound code.
- That the firewall beats a plain LLM review at seed detection.
- That specs make architecture worse (the reversal runs through size).
- Any SO5 claim beyond one vendor, two tasks and single-shot generation.

[Docs/reports/director-report-2026-10.md §1, "What this supports" and "What it does not support yet"; §6]

Threats to validity and limitations

Table 6. Main threats, grouped by validity type, with how Study 1 reports them and what v1.3 changes.

Type	Threat	Treatment
Construct	Recall is construct sensitivity: the queries match the canonical construct the same author wrote. Twin specificity holds by construction.	Stated as such; v1.3 has a separate model re-derive expected findings blind
	The judge's catch rate is unmeasured: test B placed 0 seeds.	v1.3 adds realistic anti-pattern seeds
Internal	Corrections made after seeing data (ADR-025, ADR-026, ADR-033).	Each is an ADR, shown beside the registered reading
	E7 specs generated mechanically from folder names; labels from a single-model panel, author audit pending.	v1.3 greenfield: the spec is the intent by construction; 30-item audit sealed
External	Held-out set of 5 NestJS, 1 Clean and 1 Layered bases. SO5: one vendor, two tasks, single-shot generation. FLOWS_TO empty on real code.	v1.3 per-style reference projects; Study 2 on fresh sealed tasks
Conclusion	N = 75 below the floor of 80; real-code precision on 24 items (n_{eff} 11.5); LLM-review pilot stopped at 25 of 42 reviews.	Reported as a deviation and as pilots, not estimates

[Docs/threats-to-validity.md, TV-06, TV-88, TV-89, TV-96, TV-109, TV-110; Docs/analysis-plan.md §10 B1, B2; results/labels/live-call-ledger.md; Docs/v1.3/validation-protocol.md]

Contributions of the thesis so far

C1	**Spec-to-check compiler.** A YAML architecture spec compiles to 27 Cypher fitness functions; Clean, Layered and NestJS libraries at 27/27, with Hexagonal and DDD added in v1.3.
C2	**Architectural Property Graph.** Type-checker edges (injections, calls) carry the pattern rules: without them FF-P02 and FF-P04 go blind on real bases.
C3	**A pre-registered, hash-gated evaluation.** v25 with 85 artefacts, every post-hoc change an ADR, 310 result tables reproduced byte for byte.
C4	**Evidence on specs and LLM agents (E1).** Specs change the structure agents build; at equal size a full AaC spec beats minimal prose (Cliff's δ 0.52).
C5	**A bounded result on the LLM judge.** Consistent (AC1 0.982) and model-stable (κ 0.766), yet with no verdict effect at the registered 8% weight.
C6	**A per-link validity protocol (v1.3)** that tests code → graph, spec, rules and meaning separately before Study 2.

IV

v1.3 validity cycle

The four-link measurement chain · Study 1, v1.3 and Study 2 · First evidence

Before Study 2: prove each link of the measurement chain on greenfield code

LINK 1

Code → graph

Is the graph faithful? Checked against dependency-cruiser, the TypeScript compiler and hand-written graphs.

LINK 2

Spec

In greenfield the spec is the intent by construction. Brownfield inference (ISE) comes later.

LINK 3

Rules

Recall and precision per style and per rule on reference projects with known truth.

LINK 4

Meaning

Is each finding *correct* (mechanical) and *meaningful* (labelled, author-audited)?

Table 7. The three stages of the evaluation.

Study 1 (v1.2E)	v1.3 (development)	Study 2
Frozen, reported exactly as run	May learn from all Study 1 data; every change is an ADR	Pre-registered before it runs, on fresh sealed tasks: firewall-in-the-loop vs no firewall, over multi-step feature builds

Targets (provisional): graph ≥ 0.95, recall ≥ 0.90 per style, blocking-rule precision ≥ 0.90, judge catch rate ≥ 0.80. A missed target is either fixed or waived in writing.

[Docs/v1.3/validation-protocol.md; Docs/v1.3/rule-tiers.md; Docs/v1.3/sealed-hashes.md]

First evidence: the graph is faithful, and one real defect was found and fixed

99.2–99.7%

Edge agreement with the TypeScript compiler and dependency-cruiser on the 72 E1 projects, with 0 unexplained extractor bugs

Barrels

Imports through re-export files were routed past the barrel. Fix (ADR-V13-01): record both the literal and the resolved edge.

71 cases

Reference projects for Clean, Layered, NestJS, Hexagonal and DDD, each with known truth and near-miss controls

- **New styles:** Hexagonal (Cockburn, 2005) and DDD, tactical and strategic (Evans, 2003; Vernon, 2013). Every rule is traced to its source (Appendix A3).
- **Anti-circularity:** a separate model re-derives the expected findings blind, and the author adjudicates disagreements.
- **Rule tiers sealed in advance:** only structural style constraints can block, and proxies and conventions only warn. The Study 2 tasks are sealed by hash.
- An adversarial review (2 blocking, 5 major findings) shaped the protocol before any measurement.

[PR #81, Docs/DiagnosticRuns/v1.3-graph-fidelity-2026-10-14.md (branch v1.3); Docs/v1.3/adr-v13.md, ADR-V13-01; PRs #82–#84; Docs/v1.3/rule-tiers.md; Docs/v1.3/sealed-hashes.md; Docs/v1.3/style-rule-sources.md]

V

Decisions

Direction · Reporting and scope

What I need from you

Direction

- Approve the frame: Study 1 as the pilot, v1.3 as validation, Study 2 as the main result.
- Confirm the greenfield focus, with brownfield spec inference as a later chapter.
- Confirm the v1.3 exit targets, or adjust them.

Reporting and scope

- How to frame the reversed E1 spec effect and the Codex arm.
- Would you rate 30 items as an independent second human rater?
- Paper split (E1 study, tool, judge) and which comes first.
- Extensions: Grok arm, MCP firewall-in-the-loop as the Study 2 treatment.

> Full evidence, with a source path for every number: `Docs/reports/director-report-2026-10.md`, and the v1.3 brief in Notion.

[Docs/reports/director-report-2026-10.md §7]

References

Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. Wiley.

Cockburn, A. (2005). *Hexagonal architecture* (ports and adapters). alistair.cockburn.us.

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

Ford, N., Parsons, R., & Kua, P. (2017). *Building Evolutionary Architectures: Support Constant Change*. O'Reilly Media.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

Gwet, K. L. (2008). Computing inter-rater reliability and its variance in the presence of high agreement. *British Journal of Mathematical and Statistical Psychology, 61*(1), 29–48.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.

Software baselines: dependency-cruiser 18.4.0 with typescript 5.9.3; Neo4j 5.26 community, APOC 5.26.24; ts-morph. Thesis evidence: `Docs/reports/director-report-2026-10.md` (branch `v1.2e`, merge commit `b703468`).

A

Appendix (backup)

A1 Detailed SO4 table · A2 Pre-registration bumps · A3 Canonical sources for the blocking rules

A1. SO4 held-out, symbolic-only: all four readings

Table A1. Primary interval: cell-percentile bootstrap over (project, operator) cells. Precision is seeded differential precision.

Reading	Status	N	TP	FP	FN	Recall (95% CI)	Precision (95% CI)	F1 (95% CI)	Twin spec.
Strict: instrument v1, rule 1.1.0	pre-registered	75	67	13 (strict)	8	0.893 (0.787 to 0.974; 40 cells)	0.838 (0.719 to 0.970; 36 cells)	0.865 (0.784 to 0.943)	60/73
Corrected: instrument v1, rule 1.2.0 plus style guard	post-hoc ADR-025	73	67	0	6	0.918 (0.818 to 1.000; 39 cells)	1.000 (bootstrap degenerate; Clopper-Pearson if independent 0.946 to 1.000)	0.957 (0.900 to 1.000)	68/73
Labelled: instrument v1, guarded case, rule-1.1.0 FP set adjudicated by the P1 labels (MAT-10)	post-hoc ADR-033; labelling procedure registered	73	67	3 (labelled; the 3 `uncertain` items)	6	0.918 (0.818 to 1.000; 39 cells)	0.957 (0.893 to 1.000; 36 cells; Wilson if independent 0.881 to 0.985)	0.937 (0.878 to 0.986)	60/73
v2: instrument v2, rule 1.2.0 plus style guard	post-hoc ADR-026	73	67	0	6	0.918 (0.818 to 1.000)	1.000	0.957 (0.900 to 1.000)	68/73

Table A2. Recall by stratum.

Stratum	Corrected recall (95% CI)	Strict recall
Core (4 bases, seen in development)	0.950 (0.846 to 1.000)	0.950
E7 round 1 (3 bases, the only unseen stratum)	0.879 (0.697 to 1.000; 17 cells)	0.829 (0.647 to 1.000)
In coverage	1.000 (36 cells)	0.971

The 10 type-check losses: MO-SO02 5, MO-X01 2, MO-P01 1, MO-SO01 1, MO-DF01 1, diagnosed as infeasible sites (ADR-024). The labelled reading replaces a withdrawn N = 75 row (precision 1.000, F1 0.944; ADR-033).

[results/so4-heldout/agg-strict/prf_overall.csv; results/so4-heldout/score-strict.json; results/so4-heldout/agg-corrected/prf_overall.csv; results/so4-heldout/agg-corrected/twins.csv; results/so4-heldout/score.json; results/so4-heldout/agg/prf_overall.csv; results/so4-heldout/v2/agg/prf_overall.csv; results/so4-heldout/agg-strict/golden_instances.csv]

A2. The pre-registration bumps that matter for Study 1

Table A3. Bumps v2 to v11.

Bump	Version	What it registered
P-1	v2	catalogue frozen, k = 2
P-M	v3	methodology corrections, ADR-020
P-U6	v4	U6 integration unit
P-4	v7	judge freeze
P-SO4	v8	seeded SO4 entries
(ADR-025)	v9	matching rule 1.2.0
(ADR-026)	v10	instrument v2
(ADR-027)	v11	E7 extension

Table A4. Bumps v14 to v25.

Bump	Version	What it registered
P-A28	v14	proportional judge variant
P-A28R	v15	review fixes to the variant
P-JW	v17	judge-weighted variant
P-CX	v18	Codex arm
P-TC	v19	tool comparison
P-TS	v20	E1 tsconfig fix
P-AUD	v21	reproduction-audit corrections (post-hoc)
P-LR	v24	plain LLM-review baseline pilot, exploratory (ADR-036)
P-JR	v25	judge model robustness check, registered before any Sonnet call (ADR-038)

[corpus/prereg.json; Docs/prereg-reasons.md; ADR-025, ADR-026, ADR-027, ADR-033, ADR-036, ADR-038]

A3. Canonical source for each v1.3 blocking rule

Table A5. Constraints are paraphrased with a chapter reference in the source file; page-exact quotes are pending the author's check against the books.

Style	Rule	Source
Clean	Dependency direction (FF-S01)	Martin (2017), ch. 22 "The Clean Architecture", the Dependency Rule
Clean	Domain purity (FF-P01)	Martin (2017), ch. 22; ch. 32 "Frameworks are details"
Clean	Repository / gateway placement (FF-P03)	Martin (2017), ch. 22 (interface adapters); ch. 11 (DIP)
Clean, Layered	Acyclic dependencies (FF-S02)	Martin (2017), ch. 14 "Component Coupling", ADP
Layered	Downward-only, no skipping when strict (FF-S01, FF-S03)	Fowler (2002), ch. 1 "Layering"; Buschmann et al. (1996), Layers pattern
Hexagonal	Core isolation, dependency direction (FF-S04, dependency-direction-graph, core-isolation)	Cockburn (2005)
Hexagonal	Port-adapter (port-adapter-pattern)	Cockburn (2005)
DDD	Aggregate boundary	Evans (2003), ch. 6 "Aggregates"; Vernon (2013), ch. 10
DDD	Repository only for aggregate roots	Evans (2003), ch. 6 "Repositories"; Vernon (2013), ch. 12
DDD	Value-object immutability	Evans (2003), ch. 5 "Value Objects"; Vernon (2013), ch. 6
DDD	Isolated domain (framework-free)	Evans (2003), ch. 4 "Isolating the Domain"
DDD	Bounded-context isolation	Evans (2003), ch. 14 "Maintaining Model Integrity"; Vernon (2013), ch. 2–3
NestJS	Module boundaries and DI (style preset)	NestJS documentation, "Modules", "Providers"

[Docs/v1.3/style-rule-sources.md (2026-10-14, Fable review finding 6); sealed in Docs/v1.3/sealed-hashes.md]